

Intro to AI Assignment 1

Huzaifa Iqbal (31563)

Spring 2026

Introduction

This report explores the formulation and evaluation of the word ladder problem within a continuous semantic embedding space using 100 dimensional GloVe vectors. Unlike classic word ladders that rely on character edits, this approach defines transitions based on semantic similarities, where each state represents a word and actions move between neighboring points in the vector space. By implementing and comparing uninformed search algorithms such as BFS, DFS, UCS against informed strategies such as Greedy Best-First, A^* , we will analyze the efficiency of navigating large state spaces. This assignment demonstrates how heuristic search can effectively identify paths between disparate concepts through mathematical similarity.

Task 1: Problem Formulation

1.1 State Representation

A state is defined as a unique word w existing within the provided GloVe vocabulary of 20,000 words. Each state is mapped to a 100-dimensional vector $v \in R^{100}$ representing its position in the semantic embedding space.

1.2 Initial State and Goal Test

- **Initial State:** The starting word w_{start} provided by the user.
- **Goal Test:** A check to determine if the current word $w_{current}$ is identical to the target word w_{goal} .

1.3 Action Space

Actions are defined as moving from the current word to one of its k nearest neighbors in the embedding space. For this implementation, we choose $k = 20$. **Justification:** A value of $k = 20$ provides enough connectivity to avoid frequent dead ends while maintaining a manageable branching factor for uninformed search algorithms.

1.4 Path Cost Function

We utilize a Uniform Cost Function where each transition between words has a cost of 1. **Justification:** This ensures that the search finds the path with the minimum number of semantic "jumps," resulting in the most concise word ladder.

1.5 Heuristic Function

We define the heuristic $h(n)$ as the Cosine Distance between the current word vector u and the goal word vector v :

$$h(n) = 1 - \frac{u \cdot v}{\|u\| \|v\|}$$

Intuition: Since similar words have high cosine similarity (approaching 1), their cosine distance will approach 0, correctly guiding the search toward the goal.

Task 2: Search Algorithm Implementation

2.1 Implicit Graph Construction

We have not pre defined the search space as an explicit graph. Instead, the graph is constructed implicitly during the expansion. Each state s represents a word from the 20,000 word vocabulary. For any given node, the transition model generates k successors by identifying the k nearest neighbors in the 100 dimensional GloVe embedding space.

2.2 State and Node Representation

State and Node are 2 separate things:

- **State:** Represents the current word (e.g., "apple").
- **Node:** A data structure containing the state, a pointer to the *parent* node, and the path cost $g(n)$.

2.3 Uninformed Search Algorithms

I implemented Three uninformed search strategies to find a path from the initial word to the goal word:

1. **Breadth-First Graph Search (BFS):** Explores the shallowest nodes first using a FIFO queue. An explored set is used to avoid cycles.
2. **Depth-First Graph Search (DFS):** Explores the deepest nodes first using a LIFO stack. To ensure that the code terminates in the large implicit state space, a depth limit, $L = 50.$, was implemented as a safeguard.

3. **Uniform Cost Search (UCS):** Prioritizes nodes based on the cumulative path cost $g(n)$. In this problem, the edge cost between any two semantic neighbors is defined as:

$$\text{step_cost}(\text{currentstate}, \text{action}, \text{nextstate}) = 1$$

This ensures that UCS finds the path with the minimum number of transitions.

2.4 Informed Search Algorithms

Informed search uses a heuristic function $h(n)$ to perform search. The heuristic used is the Cosine Distance between the current word vector $\vec{u}$ and the goal word vector $\vec{v}$:

$$h(n) = 1 - \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|} \quad (1)$$

1. **Greedy Best-First Search:** Expands nodes by solely relying on the heuristic value: $f(n) = h(n)$. This algorithm prioritizes semantic proximity to the goal but it does not guarantee shortest path.
2. **A* Search:** Expands node by combining path cost and heuristic estimates:

$$f(n) = g(n) + h(n)$$

By balancing the distance already traveled (g) with the estimated distance to the goal (h), A* search achieves both optimality and efficiency.

Task 3: Experimental Evaluation

The table provided in this section consolidates the performance metrics for five search algorithms across five distinct word pairs selected from the 20,000 word GloVe subset. All tests were conducted with $k = 20$ and a DFS depth limit of $L = 50$.

3.1 Comparative Analysis

Analysis of the multi-pair data reveals consistent behavioral patterns across the semantic state space:

- **Heuristic Effectiveness:** In almost all the word pairs, A* Search significantly outperformed DFS and UCS in number of nodes expanded. This demonstrates that when the Cosine Distance heuristic accurately reflects the semantic path, informed search provides substantial efficiency gains.

Word Pair	Algorithm	Nodes Expanded	Time (s)	Path Length
<i>panah</i> → <i>haloze</i>	BFS	873	8.7713	5
	DFS	6452	12.8758	50
	UCS	2478	19.2062	5
	Greedy	6	0.0494	7
	A*	1639	13.9768	5
<i>kazuhito</i> → <i>yasunobu</i>	BFS	6	0.0264	3
	DFS	1637	1.4614	50
	UCS	89	0.3733	3
	Greedy	2	0.0093	3
	A*	21	0.0958	3
<i>doped</i> → <i>stojic</i>	BFS	5772	42.3536	6
	DFS	11943	19.6475	50
	UCS	14816	107.1579	6
	Greedy	652	4.8553	14
	A*	9528	71.2399	6
<i>qianmen</i> → <i>nurturance</i>	BFS	4438	17.5801	7
	DFS	8458	7.1129	50
	UCS	9699	40.8581	7
	Greedy	18	0.0809	14
	A*	8978	38.0785	7
<i>priebe</i> → <i>gelfand</i>	BFS	8353	34.6472	10
	DFS	14524	14.6153	50
	UCS	12694	65.3409	10
	Greedy	213	0.9384	32
	A*	10515	48.4907	10

Table 1: Consolidated Performance Comparison across Multiple Word Pairs

- **The Greedy Gap:** Across all pairs, Greedy Best First Search remained the fastest algorithm but consistently failed to find the shortest path. In almost all pairs, it was not able to provide the shortest path possible, instead we can see shorter paths in BFS, UCS and A*, illustrating its susceptibility to local optima in vector space.
- **DFS and Completeness:** DFS consistently expanded a high number of nodes and returned paths equal to the depth limit. This confirms that without the radial exploration of BFS or the goal guidance of A*, DFS moves through the 100 dimensional space inefficiently.
- **Search Effort vs. Optimality:** UCS and BFS consistently matched path lengths, but UCS exhibited higher runtimes due to the use of priority queue as compared to the FIFO queue used in BFS.

Task 4: Analysis and Discussion

4.1 Uninformed vs. Informed Search Efficiency

Uninformed search algorithms, such as BFS and UCS, explore the state space in a shallow manner. eventhough this guarantees the discovery of the shortest path, it often results in high node expansion counts. In contrast, informed searches like A* use heuristics to prioritize paths toward the goal.

A clear contrast in search efficiency is observed when comparing the informed A* algorithm against the uninformed UCS and DFS strategies. In the *kazuhito* $\rightarrow$ *yasunobu* test case, A* reached the goal by expanding only 21 nodes. In comparison, UCS expanded 89 nodes to find the same optimal path, while DFS expanded 1,637 nodes. This demonstrates that for semantic wise closely related words, A* provides a 76% reduction in search effort compared to UCS.

However, A* is not always efficient. In the *doped* $\rightarrow$ *stojic* case, A* expanded 9,528 nodes and eventhough this is more efficient than UCS (14,816 nodes) and DFS (11,943 nodes), the high expansion count suggests that the heuristic was partially deceptive. In this area of the state space, A* was forced to explore semantic wise, dense and technical terms related to 'doped' before finding the path to the goal. These results confirm that while A* is significantly more focused than its uninformed counterparts, its performance remains a function of the heuristic's accuracy relative to the graph.

4.2 Failure Case: Greedy Best-First Search

Greedy Best-First Search prioritizes immediate proximity to the goal ($h(n)$) without considering the path cost ($g(n)$). A clear failure case is observed in the *priebe* $\rightarrow$ *gelfand* pair:

- **Optimal Path Length:** 10 steps (found by BFS/A*).
- **Greedy Path Length:** 32 steps.

The algorithm became trapped in a semantics, taking 22 extra steps because it chose neighbors that looked similar to the goal but were not on the best or good path towards it.

4.3 Heuristic Admissibility and Consistency

The Cosine Distance heuristic is generally admissible because its values range from $[0, 2]$, while the actual cost $h^*(n)$ to reach the goal is an integer ≥ 1 . Since $h(n)$ rarely overestimates the number of steps required to reach the goal in a high dimensional space, it satisfies the condition $h(n) \leq h^*(n)$.

The heuristic is also consistent. In the vector space, the difference in cosine distance between two neighbors is ususally less than or equal to the cost of the edge between them (1), satisfying the triangle inequality $h(n) \leq c(n, a, n') + h(n')$, where n is the current state, a is the action, n' is the next state.

4.4 Misleading Semantic Similarity

Semantic similarity is misleading when the vector space contains groups that are semantically close but connection wise distant from the goal. In the *doped* $\rightarrow$ *stoic* test, A* expanded nearly 4,000 more nodes than BFS. This occurred because "doped" has many technical neighbors that have high cosine similarity but do not lead to the goal "stoic". A* exhausted these misleading groups before backtracking, whereas BFS's blind expansion reached the goal more directly.

4.5 Impact of the Neighbor Factor (k)

The choice of k directly determines the branching factor of the search tree:

- **Small k :** Reduces memory and time complexity but risks incompleteness. If k is too low, the graph may become disconnected, making the goal unreachable from the start state.
- **Large k :** Increases the likelihood of finding shorter paths but causes an exponential increase in the Nodes Expanded. As seen in our UCS results (reaching 107s for $k = 20$), higher values of k can make uninformed search computationally difficult.

Task 5: GUI Implementation

5.1 System Features and Controls

The GUI is structured into two main components: an input sidebar and a results dashboard.

- **Input Controls:** The sidebar provides text fields for the Start and Goal words, a dropdown menu to select among the five search algorithms (BFS, DFS, UCS, Greedy, and A*), and a slider to adjust the neighbor factor (k).
- **Resource Optimization:** To handle the 20,000 word GloVe dataset efficiently, the application utilizes the `@st.cache_resource` decorator. This ensures the 100 dimensional vector matrix is loaded into memory only once.
- **Error Handling:** The system includes a validation layer that checks if the input words exist in the dictionary. If a word is missing, the GUI terminates the search process and notifies the user through an error message.

5.2 Output and Metrics Display

Upon execution, the GUI dynamically renders the results of the search. The output display includes:

- **Path Visualization:** The successful word ladder is displayed as a sequence of semantic transitions.
- **Metrics:** Real time data for path length, nodes expanded, and execution time are displayed.

Application Perspective

The ability to identify semantic paths in a high dimensional embedding space has significant implications for real world domains where conceptual relationships are more valuable than literal keyword matching.

6.1 Information Retrieval and Discovery

In modern information retrieval, semantic search paths can bridge the vocabulary gap between users and content. By navigating the vector space, a search engine can suggest a sequence of related topics that connect a niche query to a broader knowledge base. This allows researchers to find relevant documents that use different terminology but reside in the same semantic neighborhood.

6.2 Medical Informatics and Research

In the medical domain, semantic paths can assist in drug repurposing and diagnostic discovery. By analyzing embeddings of medical literature, a system could identify a path between a specific chemical compound and a biological symptom that are not explicitly linked in a single study. These semantic bridges can uncover hidden connections in pharmacological data, potentially highlighting new applications for existing treatments or identifying rare disease correlates.

6.3 Explainable Recommendations

Beyond simple discovery, these paths provide a layer of interpretability. A recommendation system in a domain like e-commerce or digital media can present a semantic path to explain the conceptual journey between a user's known preferences and a new recommendation, which would increase user trust and engagement.

Conclusion

This assignment successfully demonstrates that while informed search algorithms like A* are theoretically superior due to their use of semantic heuristics, their real world efficiency is highly dependent on the structure of the word embedding graph. Our experiments confirmed that deceptive semantic groups can occasionally lead informed searches to expand more nodes than radial uninformed searches. The integration of GloVe embeddings with classical search

techniques proves to be a powerful tool for navigating relationships between words and languages. The developed GUI further provides an accessible platform educational visualization of these AI principles.